

Ride-Sharing System Implementation Report

AI Assistant

October 21, 2024

Contents

1	Overview	1
2	Server Implementation (server.py)	2
2.1	Main Components	2
2.2	Data Structures	2
2.3	APIs Implemented	2
2.3.1	Rider APIs	2
2.3.2	Driver APIs	2
3	Rider Client Implementation (rider.py)	3
3.1	Main Component	3
3.2	Data Structures	3
3.3	APIs Used	3
4	Driver Client Implementation (driver.py)	3
4.1	Main Component	3
4.2	Data Structures	3
4.3	APIs Used	3
5	Key Functionalities	3
5.1	Load Balancing	3
5.2	Timeout Handling	4
5.3	State Management	4
5.4	Concurrent Request Handling	4
6	Areas for Improvement	4
7	Conclusion	4

1 Overview

The system consists of three main components:

1. Server (server.py)
2. Rider Client (rider.py)
3. Driver Client (driver.py)

The communication between these components is facilitated using gRPC, with the API definitions specified in rider.proto and driver.proto files.

2 Server Implementation (server.py)

2.1 Main Components

- **UberServer**: The main server class that initializes and runs the gRPC server.
- **RiderClientServicer**: Handles rider-related requests.
- **DriverClientServicer**: Handles driver-related requests.
- **LoggingInterceptor**: Provides logging functionality for incoming requests.

2.2 Data Structures

1. `request_buffer`: A list storing ride IDs of pending requests.
2. `ride_reqs`: A dictionary mapping ride IDs to `RideRequest` objects.

2.3 APIs Implemented

2.3.1 Rider APIs

1. `RequestRide`: Creates a new ride request and adds it to the `request_buffer`.
2. `GetRideStatus`: Retrieves the current status of a ride.

2.3.2 Driver APIs

1. `AmAvailable`: Assigns an available ride to a driver.
2. `AcceptRide`: Marks a ride as accepted by a driver.
3. `RejectRide`: Handles ride rejection by a driver.
4. `TimedOutRide`: Handles cases where a driver doesn't respond in time.
5. `CompleteRide`: Marks a ride as completed.

3 Rider Client Implementation (rider.py)

3.1 Main Component

- **RiderClient**: Handles rider interactions with the server.

3.2 Data Structures

1. **conns**: A dictionary storing connection information for multiple servers.
2. **cur_ride_resp**: A list storing current ride information [ride_id, driver, status].

3.3 APIs Used

1. **request_ride**: Sends a ride request to the server.
2. **request_ride_status**: Retrieves the status of the current ride.

4 Driver Client Implementation (driver.py)

4.1 Main Component

- **DriverClient**: Handles driver interactions with the server.

4.2 Data Structures

1. **conns**: A dictionary storing connection information for multiple servers.
2. **cur_ride_req**: A tuple storing current ride request information (ride_id, rider_id, pickup_location, destination, status).

4.3 APIs Used

1. **am_available**: Checks for available ride requests.
2. **accept_ride**: Accepts an assigned ride.
3. **reject_ride**: Rejects an assigned ride.
4. **complete_ride**: Marks a ride as completed.

5 Key Functionalities

5.1 Load Balancing

The system implements a simple round-robin load balancing strategy. Both rider and driver clients maintain a list of server ports and cycle through them for each request.

5.2 Timeout Handling

The driver client implements a timeout mechanism using a separate thread (`timeout_tracker`). If a driver doesn't respond to a ride request within the specified timeout period, the ride is automatically rejected.

5.3 State Management

The server maintains the state of ride requests using the `ride_reqs` dictionary. This allows for efficient lookup and updates of ride information.

5.4 Concurrent Request Handling

The server uses `ThreadPoolExecutor` to handle multiple requests concurrently, allowing it to serve multiple riders and drivers simultaneously.

6 Areas for Improvement

1. Implement more sophisticated load balancing strategies.
2. Add authentication and security measures.
3. Implement persistent storage for ride information.
4. Add more error handling and edge case management.
5. Implement a more robust system for handling server failures and request retries.

7 Conclusion

The implemented ride-sharing system provides a solid foundation for handling basic ride requests, driver assignments, and ride completions. The use of gRPC allows for efficient communication between clients and servers, while the chosen data structures enable quick access and updates to ride information. With further enhancements in areas such as security, persistence, and fault tolerance, this system could be scaled to handle more complex ride-sharing scenarios.